

Profiling and Optimizing the AssetOpsBench Plan-Execute Pipeline

Built adaptive routing systems that selectively enables reasoning mode, recovering most of the accuracy gains of always-on reasoning while saving up to 1.22 seconds per query.

Team 21

Shen Li · sl6008

Charles Xu · tx2263

Ann Li · acl2246

Caroline Cahill · clc2240

Mentor: Dhaval Patel

GitHub: <https://github.com/jasonlee-1024/AssetOpsBench/tree/main>

Tracking:

<https://wandb.ai/ccahill19-columbia-university/hpml-semester-project/reports/DistilBERT-Classifer-Training--VmlldzoxNjgwNDk5Ng>

Motivation & Problem

Why this matters

- The real-world setting of our project is industrial asset operations and maintenance
- Cost / latency
 1. Reasoning models are increasingly available as drop-in replacements for standard LLMs, but their additional inference cost in the context of tool-augmented pipelines remains uncharacterized. It is unknown whether the accuracy gains justify the latency overhead for industrial agent tasks.
 2. Cost of spawning MCP servers as on-demand stdio subprocesses per tool call may introduce significant overhead at scale
 3. Latency breakdown of the pipeline is currently unknown. It is unclear how much time is spent in LLM calls vs MCP tool execution vs network & process overhead
- A naive choice to use thinking mode or not in the planning phase is not good enough because enabling thinking mode introduces extra costs that are wasted on simple tasks, even though it does significantly increase accuracy on complex tasks.

Problem statement

Goal: This project provides the first systematic performance characterization of the AssetOpsBench plan-execute pipeline, compares Gemma 4 26B with and without thinking mode to quantify the latency and accuracy tradeoff of whether thinking mode is worthwhile for industrial asset operations tasks, and implements a rules-based router and a DistilBERT Classifier to label and route complex tasks to a thinking-enabled planner and simple tasks to a standard planner.

Technical Approach — Model & Data

Model

- Google Gemma 4 26B
- Thinking vs. non-Thinking
- AssetOpsBench's plan-execute pipeline as agent.

Dataset

We used the [AssetOpsBench dataset from IBM Research](#) with an apache 2.0 license. We used the train split of the “scenarios” subset, which contains 152 scenarios.

We select 40 scenarios for profiling.

Hardware

- **NVIDIA A100 GPU**
- Software stack:
- **vLLM 0.19.0 as the inference server,**
- PyTorch 2.4.0,
- CUDA 12.4, and Python 3.11.

Technical Approach — Optimizations Applied

1. Selective-Thinking in Plan–Execute Pipeline

Thinking is more important at the planning stage; execution is mostly mechanical tool-calling, so we only enable thinking in plan stage.

2. Dynamic-Thinking , Rules-Based Router

Many requests carry obvious signals of difficulty — match them with hand-written rules and dynamically determine whether need thinking.

3. Dynamic-Thinking, DistilBERT Classifier

Let a small trained classifier learn the "needs thinking?" decision boundary from data instead of guessing it with rules.

Profiling Methodology

Tools

- Weights & Biases for run-level tracking
- vLLM logs & customized metrics
- LLM as a judge

Accuracy Metrics

- **Task Completion**
- Data Retrieval Accuracy
- Generalized Result Verification
- Agent Sequence Correctness
- Clarity and Justification
- Absence of Hallucinations

Performance Metrics

System Level

- **End-to-end latency** (`runner.run()` wall time)
- **Planning latency** (`planner.generate_plan()` wall time)
- Arg resolution latency (`_resolve_args_with_llm()`)
- Summarization latency (summarization call wall time)
- MCP tool execution time (`_call_tool()` per server)

Hardware Level

- Time-to-first-token (vLLM /metrics)
- Token throughput (tok/s) (vLLM /metrics)
- GPU memory utilization (vLLM /metrics)

Results — Initial Profiling Findings

21.5%

greater latency cost

[no thinking mode → thinking mode]

16%

increase in task completion

[no thinking mode → thinking mode]

51%

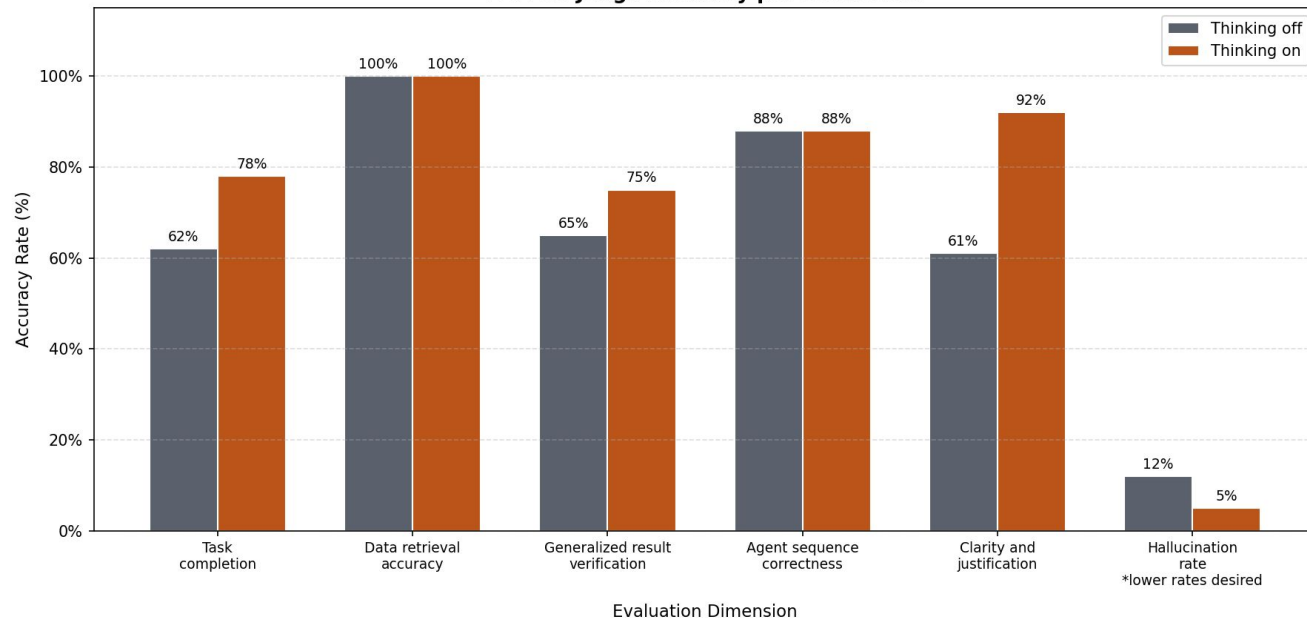
of total time spent in *planning* phase

[thinking mode]

All measurements taken over 1 run of 40 scenarios on NVIDIA A100 80GB (RunPod).

Results — Initial Accuracy Across 6 Metrics

LLM-as-Judge Accuracy per Dimension



Takeaway

- This chart shows the accuracy rates across our six evaluation dimensions when thinking mode is on compared to when it is off
- Using thinking mode most improved our Clarity and Justification metric, increasing it from 61% accuracy to 92%
- Enabling thinking mode did not at all affect Data Retrieval accuracy or Agent Sequence Correctness

Results — Post Optimization Findings vs. Always Off

*[always off →
rule-based router]*

2.33s

slower inference

8pp

higher task completion

21pp

higher response clarity

*[always off →
classifier model]*

2.26s

slower inference

11pp

higher task completion

24pp

higher response clarity

All measurements taken over 1 run of 40 scenarios on NVIDIA A100 80GB (RunPod).

Results — Post Optimization Findings vs. Always On

*[always on →
rule-based router]*

1.22s

faster inference

8pp

lower task completion

10pp

lower response clarity

*[always on →
classifier model]*

0.98s

faster inference

5pp

lower task completion

7pp

lower response clarity

All measurements taken over 1 run of 40 scenarios on NVIDIA A100 80GB (RunPod).

Profiler Evidence

```
• $ python scripts/bench_latency.py --model-id Google/Gemma4-26B --batch-size 5 --thinking false

Loading scenarios from HuggingFace: ibm-research/AssetOpsBench (split=train) ...
Loaded 141 scenarios from ibm-research/AssetOpsBench

Model:      Google/Gemma4-26B
Thinking:    disabled
ID range:    40 scenarios
Batch size:  5
Grade mode:  custom

[1/40] What is the current operational status of chiller CH-01 at site MAIN?
  expected: The agent should retrieve asset status for CH-01 and report whether it is running, idle, or faulted.
  latency: plan=6.21s execute=4.93s summarize=3.54s total=14.68s
  answer: Chiller CH-01 at site MAIN is currently operational (running). The latest telemetry shows supply water temperature at
6.8°C, return water temperature at 12.3°C, and compressor load at 74%. No active faults detected.
  grading...
  graded: {'task_completion': True, 'data_retrieval_accuracy': True, 'generalized_result_verification': True,
'agent_sequence_correct': True, 'clarity_and_justification': False, 'hallucinations': False}

... (39 more scenarios omitted) ...

Summary over 40 scenarios (40 graded)

Plan:      6.605s  (43.8%)
Execute:    4.677s  (31.0%)
Summarize:  3.800s  (25.2%)
Total:      15.082s
— Accuracy per dimension —
  task_completion: 62% passed
  data_retrieval_accuracy: 100% passed
  generalized_result_verification: 65% passed
  agent_sequence_correct: 88% passed
  clarity_and_justification: 61% passed
  hallucinations: 12% hallucinated

Full results saved to scripts/bench_results.json
```

```
• $ nvidia-smi -l 1

Tue Apr 15 16:15:03 2026
+-----+
| NVIDIA-SMI 550.54.15                  Driver Version: 550.54.15   CUDA Version: 12.4   |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                               Temp  | Bus-Id      Disp.A | Volatile Uncorr. ECC |
| Fan  Speed  Perf            Pwr   | Memory-Usage GPU-Util| Compute M.  MIG M. |
+-----+-----+-----+-----+-----+-----+
|  0  NVIDIA A100-SXM4-80GB  78C  | 00000000:00:04.0  Off  |           0          |
| N/A   N/A    P0              398W | 80912MiB / 81920MiB | 99%    Default      |
|                                     |           |           Disabled   |
+-----+-----+-----+-----+-----+-----+

+-----+
| Processes:                                     |
| GPU   GI    CI          PID    Type    Process name                        GPU Memory |
| ID    ID    ID                                  Usage                        |
+-----+-----+-----+-----+-----+-----+
|  0     N/A  N/A         21847    C      python scripts/bench_latency.py     80902MiB |
+-----+-----+-----+-----+-----+-----+
```

Demo / Visualization

Lessons Learned

What worked

- Since we showed that latency concerns are concentrated in the planning phase of AssetOpsBench's plan-execute pipeline, we could target our optimization to the planning phase
- Enabling thinking mode in the planning phase produced improvements in four of our six evaluation dimensions
- Routing queries to a planning phase with or without thinking mode based on their complexity is a successful solution to the dilemma of wasted overhead from using thinking mode on simple queries that do not need it

What didn't work

- The profiling sample size of 40 scenarios to create gold labels is very small
- Our DistilBERT classifier is trained on data labeled by an LLM, not by the results of actual profiling
- Profiled and discussed the cost of spawning MCP servers as on-demand stdio processes, but did not optimize this

Next Steps

- Increase the number of scenarios we use in our experiments to produce more statistically sound results
- Train our DistilBERT classification model with real results from our profiling experiments
- Re-evaluate rule-based router with secondary rules
- Communicate with Mentor and finalize PR for contribution into open-source AssetOpsBench repository
- How does our observed accuracy-latency tradeoff in Gemma 4 26B generalize to different reasoning-capable models?

Teamwork & Contributions

Member	Primary contributions	Tools / artifacts
Shen Li	Profiling script, thinking mode integration	scripts/bench_latency.py, src/ runner.py , src/planner.py
Charles Xu	Thinking mode classifier training, model-based router	src/llm/ lmtrain.py src/llm/test_lmtrain.py
Ann Li	Rule-based router, updated proposal for mentor project resubmission	src/llm/rule_based_router.py src/llm/rule_based_router_keywords.yaml src/llm/test_rule_based_router.py
Caroline Cahill	Data-preparation script, trained thinking mode classifier on prepared data, W&B logging in classifier, presentations	src/llm/prepare_training_data.py /LLM-as-judge_results.py models/ Final presentation slides 2, 3, 7, 12, 13, Midpoint presentation slides 2, 3

Collaboration: regular bi-weekly meetings via Zoom, email thread with Mentor, shared WhatsApp group chat.

Thank you

Questions?

GitHub: <https://github.com/jasonlee-1024/AssetOpsBench/tree/main>

Tracking: <https://api.wandb.ai/links/ccahill19-columbia-university/pi21cc6x>